

John Matthew M. Perez

BSCS – 3A

PARACOMP

03/24/2026

```
$ g++ parallel_activity_openmp.cpp -o parallel_activity_openmp.exe -fopenmp

johnm@Jack MINGW64 /c/Users/johnm/OneDrive/Desktop/BSCS/PARACOMP
$ ./parallel_activity_openmp.exe
=====
Parallel Computing Activity: OpenMP C++
=====
Dataset size: 1000000 numbers
OpenMP threads available: 12

Sequential Result: 3383500000.000000
Sequential Time : 0.007667 seconds

Parallel Result : 3383500000.000000
Parallel Time   : 0.002594 seconds

Speedup        : 2.955779x
Verification    : PASS (results match)
```

```
$ g++ parallel_activity_openmp.cpp -o parallel_activity_openmp.exe -fopenmp

johnm@Jack MINGW64 /c/Users/johnm/OneDrive/Desktop/BSCS/PARACOMP
$ ./parallel_activity_openmp.exe
=====
Parallel Computing Activity: OpenMP C++
=====
Dataset size: 5000000 numbers
OpenMP threads available: 12

Sequential Result: 16917500000.000000
Sequential Time : 0.038878 seconds

Parallel Result : 16917500000.000000
Parallel Time   : 0.008181 seconds

Speedup        : 4.752160x
Verification    : PASS (results match)
```

```

$ g++ parallel_activity_openmp.cpp -o parallel_activity_openmp.exe -fopenmp

johnm@Jack MINGW64 /c/Users/johnm/OneDrive/Desktop/BSCS/PARACOMP
$ ./parallel_activity_openmp.exe
=====
Parallel Computing Activity: OpenMP C++
=====
Dataset size: 10000000 numbers
OpenMP threads available: 12

Sequential Result: 33835000000.000000
Sequential Time : 0.085737 seconds

Parallel Result : 33835000000.000000
Parallel Time : 0.025435 seconds

Speedup : 3.370792x
Verification : PASS (results match)

```

```

$ g++ parallel_activity_openmp.cpp -o parallel_activity_openmp.exe -fopenmp

johnm@Jack MINGW64 /c/Users/johnm/OneDrive/Desktop/BSCS/PARACOMP
$ ./parallel_activity_openmp.exe
=====
Parallel Computing Activity: OpenMP C++
=====
Dataset size: 20000000 numbers
OpenMP threads available: 12

Sequential Result: 67670000000.000000
Sequential Time : 0.157191 seconds

Parallel Result : 67670000000.000000
Parallel Time : 0.028736 seconds

Speedup : 5.470230x
Verification : PASS (results match)

```

Analysis:

The parallel version is faster because it divides the work among multiple CPU cores, allowing computations to happen simultaneously. The sequential version uses only one core, making it slower for large datasets. However, for smaller datasets, the speed difference is minimal due to the overhead of managing multiple threads.

Guide Questions:

1. What is the main difference between the sequential loop and the parallel loop?
 - OpenMP is faster because it uses multiple threads to process data simultaneously, but it can be slower if the overhead of thread management is higher than the computation.
2. What is the purpose of `#pragma omp parallel for`?
 - More CPU cores allow more threads to run in parallel, reducing execution time.
3. Why is reduction(`+:sum`) needed?
 - It ensures that each thread safely updates the shared variable `sum` and combines all partial results correctly.
4. What does speedup mean?
 - Speedup measures how much faster the parallel version is compared to the sequential version
5. Why do some computers get better speedup than others?
 - Computers with more CPU cores and better processing power can handle more threads simultaneously, resulting in greater speedup.